

RELATÓRIO TÉCNICO: CÁLCULO DE DISTÂNCIAS GLOBAIS

Disciplina: Pensamento Computacional e Algoritmos

Aluno: Antonio Augusto do Nascimento

1. Descrição do Problema e Regras

Este projeto teve como objetivo desenvolver um programa em Java capaz de trabalhar com um grande conjunto de dados geográficos, contendo aproximadamente 7.000 cidades ao redor do mundo. O principal desafio foi identificar quais são as duas cidades mais distantes entre si, considerando critérios de filtragem definidos pelo usuário.

As principais regras e limitações do trabalho foram:

- **Manipulação de Dados:** O programa deve ler um arquivo CSV (cidades.csv) contendo informações como Continente, País, Cidade, Latitude, Longitude e População.
 - **Restrição de Estrutura:** Não foi permitido utilizar Orientação a Objetos para representar a entidade "Cidade". Os dados deveriam ser armazenados obrigatoriamente utilizando Vetores Paralelos.
 - **Cálculo Matemático:** A distância entre as cidades deveria considerar a curvatura da Terra, utilizando a Fórmula de Haversine.
 - **Interatividade:** O usuário deve conseguir filtrar os dados por Continente, País ou População por meio de argumentos informados na linha de comando.
-

2. Instruções de Execução

O programa foi desenvolvido para ser executado pelo terminal, utilizando o redirecionamento da entrada padrão do sistema operacional para realizar a leitura do arquivo CSV.

2.1. Compilação

Antes de executar o programa, é necessário compilar o arquivo fonte:

- `javac Distancias.java`

2.2. Execução

O padrão de execução do programa é:

- `java Distancias [FILTRO] [VALOR] < cidades.csv`

A seguir estão alguns exemplos de uso:

A) Execução sem filtros (processa todas as cidades):

- `java Distancias < cidades.csv`

B) Filtro por Continente (Flag **C**):

Obs.: Para nomes com espaço, é necessário usar aspas.

- `java Distancias C Europa < cidades.csv`
- `java Distancias C "América do Sul" < cidades.csv`

C) Filtro por País (Flag **P**):

Obs.: Os nomes devem estar em inglês, conforme o arquivo CSV.

- `java Distancias P Brazil < cidades.csv`
- `java Distancias P "United States" < cidades.csv`

D) Filtro por População (Flags **+** ou **-**):

Obs.: Deve haver espaço entre o símbolo e o número.

+ → cidades com população maior ou igual ao valor informado

- → cidades com população menor ou igual ao valor informado

- `java Distancias + 10000000 < cidades.csv`
 - `java Distancias - 5000 < cidades.csv`
-



3. Representação Algorítmica (Pseudocódigo)

```
1  ALGORITMO DistanciasEntreCidades
2
3  CONSTANTES
4      RAO_TERRA = 6378.13
5      MAX_CIDADES = 10000
6
7  VARIÁVEIS GLOBAIS (Vetores Paralelos)
8
9      nomes[], latitudes[], longitudes[], populacoes[] ...
10     totalCidades = 0
11     Filtros: tipoFiltro, filtroTexto, filtroNumero
12 INICIO
13
14     // 1. Configuração dos Filtros via Argumentos
15     SE argumentos existem ENTÃO
16         tipoFiltro = argumento[0]
17         SE tipoFiltro for '+' OU '-' ENTÃO
18             filtroNumero = CONVERTER_PARA_INTEIRO(argumento[1])
19         SENÃO
20             filtroTexto = argumento[1]
21         FIM_SE
22     SENÃO
23         tipoFiltro = 'N' (Ler tudo)
24     FIM_SE
25     // 2. Leitura e Carregamento de Dados
26     ENQUANTO houver linhas no arquivo E totalCidades < MAX_CIDADES FAÇA
27         LER linha
28         SEPARAR linha em colunas
29
30         // Validação do Filtro
31         deveSalvar = FALSO
32         SE tipoFiltro == 'N' ENTÃO deveSalvar = VERDADEIRO
33         SE tipoFiltro == 'C' E continente == filtroTexto ENTÃO deveSalvar = VERDADEIRO
34         SE tipoFiltro == 'P' E pais == filtroTexto ENTÃO deveSalvar = VERDADEIRO
35         SE tipoFiltro == '+' E populacao >= filtroNumero ENTÃO deveSalvar = VERDADEIRO
36         SE deveSalvar ENTÃO
37             CONVERTER textos (lat, lon, pop) para números
38             ARMAZENAR dados nos vetores na posição [totalCidades]
39             INCREMENTAR totalCidades
40         FIM_SE
41     FIM_ENQUANTO
42
43     // 3. Processamento (Comparação Par-a-Par)
44     SE totalCidades < 2 ENTÃO
45         ESCREVER "Erro: Cidades insuficientes."
46     SENÃO
47         maiorDistancia = 0
48
49         PARA i DE 0 ATÉ totalCidades FAÇA
50             // O loop interno começa de (i+1) para evitar repetições e comparação A-A
51             PARA j DE (i + 1) ATÉ totalCidades FAÇA
52                 dist = CALCULAR_HAVERSINE(lat[i], lon[i], lat[j], lon[j])
53
54                 SE dist > maiorDistancia ENTÃO
55                     maiorDistancia = dist
56                     cidadeA = nomes[i]
57                     cidadeB = nomes[j]
58                 FIM_SE
59             FIM_PARA
60         FIM_PARA
61
62         ESCREVER "As cidades mais distantes são: ", cidadeA, " e ", cidadeB
63         ESCREVER "Distância: ", maiorDistancia
64     FIM_SE
65 FIM
```

4. Comentários e Decisões de Projeto

Para atender aos requisitos do trabalho, algumas decisões importantes foram tomadas:

- **Uso de Vetores Paralelos:** Como não era permitido utilizar classes e objetos, os dados das cidades foram armazenados em vetores separados. Todos os vetores utilizam o mesmo índice para garantir que as informações correspondam à mesma cidade.
- **Leitura via System.in:** A leitura do arquivo CSV foi feita utilizando a classe Scanner com entrada padrão (System.in), permitindo o uso do redirecionamento `<` no terminal.
- **Complexidade do Algoritmo:** O cálculo da maior distância foi realizado utilizando dois laços de repetição aninhados. O segundo laço começa em `i + 1`, evitando comparações repetidas e desnecessárias.
- **Tratamento dos Argumentos:** O programa identifica se o filtro é numérico ou textual, garantindo que os valores sejam interpretados corretamente.
- **Fórmula de Haversine:** O cálculo da distância foi implementado em um método separado, convertendo as coordenadas de graus para radianos para garantir maior precisão.

5. Dificuldades Encontradas

Durante o desenvolvimento do projeto, algumas dificuldades foram enfrentadas:

- **Controle dos Vetores Paralelos:** Foi necessário muito cuidado para manter todos os vetores sincronizados, evitando erros de associação entre os dados.
- **Uso da Linha de Comando:** No início, houve dificuldade em entender como passar corretamente os argumentos, principalmente em casos que envolviam textos com espaços e filtros numéricos.

Esses desafios contribuíram para um melhor entendimento sobre manipulação de dados, estruturas básicas e execução de programas via terminal.